

# Self-Healing and Scalable Web Infrastructure on AWS using Application Load Balancer and Auto Scaling

**Submitted by:**
KOTHA SAI LAXMAN
Roll Number: 2451-23-749-019 | Class: CSE-IoT

**Under the Guidance of:**
Mr. Krishna

**Department:**
Computer Science and Engineering (IoT)

**MVSR ENGINEERING COLLEGE**
Academic Year: 2026 – 2027

# 2. Certificate

This is to certify that the project report titled **“Self-Healing and Scalable Web Infrastructure on AWS using Application Load Balancer and Auto Scaling”** submitted by **KOTHA SAI LAXMAN (Roll No. 2451-23-749-019, CSE-IoT)** in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree has been carried out under the supervision of Mr. Krishna.

Guide Signature (Mr. Krishna)
HOD Signature
Principal Signature

# 3. Acknowledgment

I would like to express my sincere gratitude to my project guide **Mr. Krishna** for his continuous support, guidance, and encouragement throughout the development of this project.

I also thank the **Department of Computer Science and Engineering (IoT)** for providing the necessary facilities to complete this work successfully.

Finally, I thank my friends and family members for their support during the project.

# 4. Abstract

Cloud computing has revolutionized how applications and infrastructure are deployed and managed. Traditional IT infrastructures require manual monitoring and intervention to handle server failures and workload changes, which may result in downtime and poor performance.

This project presents **Self-Healing and Scalable Web Infrastructure on AWS**, which automatically detects system failures, spreads traffic across servers in different Availability Zones and dynamically adjusts computing resources based on demand. The proposed system uses Amazon Web Services such as **Amazon EC2, Application Load Balancer, Target Group, Auto Scaling Group, Amazon CloudWatch, Amazon S3, IAM and Virtual Private Cloud** to maintain application availability and performance.

The infrastructure continuously monitors system health through load balancer health checks and removes unhealthy servers from rotation, while the Auto Scaling Group replaces failed instances to ensure uninterrupted service. The system also scales resources up during traffic spikes and down during low demand, improving cost efficiency and reliability.

The implementation, carried out in the US East (Ohio) region, deploys two Nginx web servers in two Availability Zones behind an Application Load Balancer. The captured output shows both targets turning Healthy and the same load balancer address answering alternately from both servers, demonstrating high availability, fault tolerance and load balancing.

# 5. Table of Contents

1. Introduction
2. Literature Survey
3. Problem Statement
4. Objectives
5. Existing System
6. Proposed System
7. System Architecture
8. Technologies Used
9. Implementation Methodology
10. System Testing
11. Results and Discussion
12. Advantages and Limitations
13. Applications
14. Future Scope
15. Conclusion
16. References

# 6. Introduction

## 6.1 Background

Cloud computing has become one of the most important technologies in modern computing environments. Organizations rely heavily on cloud infrastructure to host applications, store data, and provide services to users worldwide.

Traditional infrastructure requires manual system management, which can lead to inefficiencies and service downtime during hardware failures or sudden increases in traffic. A single server is a *single point of failure*, and fixed capacity is either too small at peak times or wasteful at quiet times.

## 6.2 Need for the Project

Modern web applications require:

- High availability
- Automatic scaling
- Fault tolerance
- Reduced operational costs

A **Self-Healing Infrastructure** ensures that system failures are detected automatically and the failed servers are replaced without human intervention.

## 6.3 Cloud Computing Model

Three major service models include:

- **IaaS** (Infrastructure as a Service)
- **PaaS** (Platform as a Service)
- **SaaS** (Software as a Service)

This project mainly uses **Infrastructure as a Service (IaaS)**, where virtual servers (Amazon EC2), networking (VPC, load balancer) and storage (Amazon S3) are rented on demand and configured by the user.

# 7. Literature Survey

Several researchers and cloud providers have studied scalable cloud infrastructures for improving system reliability. Automated infrastructure management improves system performance and reduces downtime compared with manually managed servers. This chapter summarizes the concepts that this project builds on.

## 7.1 Load Balancing

A load balancer is placed in front of a group of servers and distributes incoming requests among them. In AWS, **Elastic Load Balancing** performs this task across multiple Availability Zones and only routes traffic to targets that pass their health checks. An *Application Load Balancer* works at Layer 7 (HTTP/HTTPS), which makes it a natural fit for web applications.

## 7.2 Auto Scaling

**Amazon EC2 Auto Scaling** keeps a desired number of instances running and changes that number according to demand. If an instance is terminated or marked unhealthy, the Auto Scaling Group launches a replacement from a Launch Template, which gives the infrastructure its *self-healing* property.

## 7.3 Monitoring and Alarms

**Amazon CloudWatch** collects metrics such as CPU utilization and raises alarms when a threshold is crossed. These alarms are the trigger for scale-out and scale-in actions, so the system reacts to load without human involvement.

## 7.4 Self-Healing Architectures

Large-scale internet companies rely on **self-healing architectures** in which failed components are detected and replaced automatically to ensure service continuity. The key ideas are redundancy (more than one server), isolation (different Availability Zones), detection (health checks) and automatic recovery (replacement of failed servers). This project applies the same principles in a simple, low-cost, student-friendly design.

# 8. Problem Statement

Traditional server infrastructures suffer from the following problems:

- **Manual system monitoring** – an administrator must watch the servers all the time and react by hand when something breaks.

- **High downtime during failures** – when the only server fails, the whole application is unavailable until it is repaired or rebuilt.
- **Limited scalability** – a fixed-size server cannot cope with sudden spikes in traffic, so users face slow responses or errors.
- **High operational cost** – servers sized for peak load stay idle most of the time, and manual maintenance consumes staff time.

These problems affect the reliability, performance and cost of web applications, especially for services that must be available around the clock.

*Therefore, there is a need for an automated cloud infrastructure that can detect failures, stop sending traffic to failed servers and adjust resources dynamically.*

# 9. Objectives

The objectives of this project are:

- **Design a scalable and highly available infrastructure** – use more than one server in more than one Availability Zone so that no single failure stops the service.
- **Distribute traffic** – place an Application Load Balancer in front of the servers so that requests are shared between them.
- **Detect and recover from failures automatically** – use health checks and an Auto Scaling Group so that unhealthy servers are removed and replaced.
- **Scale on demand** – add servers when CPU load rises and remove them when load falls.

- **Reduce manual maintenance** – build servers from a Launch Template so every instance is identical and no manual setup is required.
- **Improve availability and cost efficiency** – pay only for the capacity that is actually needed.

Together, these objectives demonstrate the core cloud ideas of *high availability*, *fault tolerance* and *elasticity*.

# 10. Existing System

Traditional hosting environments rely on a **single server** or on manually managed infrastructure. A single web server runs the application; if it crashes, runs out of memory or loses its network, the website goes offline until an administrator notices and fixes it.

Capacity is fixed in advance. If the server is small, it becomes slow or unavailable during traffic peaks; if it is large, the extra capacity is paid for even when it is idle.

## Limitations

- **No automatic failure recovery** – a failed server must be repaired or replaced manually.
- **Difficult to handle traffic spikes** – capacity cannot grow by itself when many users arrive at once.
- **High downtime risk** – one server is a single point of failure.
- **Manual monitoring required** – failures are discovered late, often by users.

These limitations motivate the proposed self-healing and scalable design described in the next section.

# 11. Proposed System

The proposed system implements **Self-Healing and Scalable Web Infrastructure** using AWS services. Instead of one server, the application runs on several identical servers behind a load balancer, and AWS services take care of detection, recovery and scaling.

## 11.1 Key Features

- **Automatic server replacement** by the Auto Scaling Group
- **Dynamic scaling** based on CPU demand
- **Load balancing** across multiple servers in two Availability Zones
- **Continuous health checking** and monitoring
- **Security** through Security Group chaining and an IAM Role instead of stored access keys

## 11.2 Working Principle

Users reach the application only through the load balancer. The load balancer forwards each request to a healthy server in the Target Group. CloudWatch watches the CPU and health of the servers; when a server fails or the load becomes high, the Auto Scaling Group launches new instances from the Launch Template and registers them in the Target Group automatically. When the load falls, extra servers are removed to save cost.

The system uses **AWS EC2 instances with Nginx, an Application Load Balancer, a Target Group, an Auto Scaling Group and CloudWatch** to achieve these capabilities. Amazon S3 stores static assets and backups.

## 11.3 Comparison with the Existing System

- **Availability** – existing: one server; proposed: two or more servers in two Availability Zones.
- **Failure handling** – existing: manual repair; proposed: health checks and automatic replacement.
- **Traffic spikes** – existing: fixed capacity; proposed: CPU-based scale-out and scale-in.
- **Cost** – existing: pay for idle capacity; proposed: pay for what is needed.

# 12. System Architecture

## Components

1. User
2. Internet
3. Application Load Balancer (listener on port 80)
4. Target Group with health checks
5. EC2 Instances running Nginx (two Availability Zones)
6. Auto Scaling Group with Launch Template
7. CloudWatch Monitoring
8. Virtual Private Cloud (VPC), subnets and Security Groups

## Architecture Diagram

*Figure 1: System architecture of the self-healing infrastructure*

**How a request travels:** the user sends an HTTP request to the load balancer DNS name. The listener forwards it to the Target Group, which selects a healthy EC2 instance in one of the two Availability Zones. Nginx on that instance returns the web page. CloudWatch collects CPU and health metrics, and its alarms make the Auto Scaling Group launch or replace instances from the Launch Template, which then register in the Target Group.

# 13. Technologies Used

| Technology | Description |
|---|---|
| AWS EC2 | Virtual servers that run the web application |
| Application Load Balancer | Public entry point; distributes traffic (Layer 7) |
| Target Group | Pool of servers; runs health checks |
| Auto Scaling Group | Maintains and scales the number of instances |
| Launch Template | Blueprint for creating identical instances |
| CloudWatch | Metrics, monitoring and alarms |

| | |
|---|---|
| VPC and Security Groups | Secure networking; HTTP only from the load balancer |
| Amazon S3 and IAM Role | Private storage; keyless access from EC2 |
| Nginx | Lightweight web server |
| Ubuntu Linux | Operating system of the instances |

These technologies were chosen because they are managed, widely used and available in the AWS free-tier friendly range, which suits a student project:

- **Managed services** – the load balancer, scaling group and CloudWatch need no server maintenance.
- **Open-source web server** – Nginx is lightweight and ideal for serving the demo pages.
- **Security by design** – Security Group chaining and the IAM Role keep the servers private and keyless.

# 14. Implementation Methodology

## Step 1: Select VPC and Subnets

Use the VPC in region US East (Ohio) us-east-2 with **two public subnets in different Availability Zones**: us-east-2a and us-east-2b. Using two zones lets the application survive the loss of one zone.

## Step 2: Create Security Groups

Create a security group for the load balancer that allows **HTTP (80)** from the internet, and a security group for the web servers that allows HTTP **only from the load balancer security group** (security group chaining).

## Step 3: Launch EC2 Instances

Launch two Ubuntu EC2 instances, **WebServer-1** in us-east-2a and **WebServer-2** in us-east-2b, and install the Nginx web server. Each page shows the server name and Availability Zone.

Example commands:

```
sudo apt update -y

sudo apt install nginx -y

sudo systemctl enable --now nginx
```

```
echo '<h1>Hello from Server 1 - AZ-A</h1>' | sudo tee
/var/www/html/index.html
```

## Step 4: Create the Target Group

Create a Target Group named **HA-Project-TG** (protocol HTTP, port 80), configure the health check, and register both instances. New targets start in the *Initial* state and become *Healthy* after passing the checks.

## Step 5: Configure the Application Load Balancer

Create an **internet-facing Application Load Balancer** named **HA-Project-ALB** with IPv4, select the two subnets, attach the load balancer security group, and add a listener on **HTTP : 80** that forwards to HA-Project-TG. Copy the DNS name of the load balancer.

## Step 6: Create the Auto Scaling Group

Create a Launch Template (AMI, instance type, security group, IAM Role and User Data that installs Nginx), then create an Auto Scaling Group from it using the same two subnets. Attach it to HA-Project-TG, enable load balancer health checks, and configure minimum, maximum and desired instances with a CPU-based scaling policy.

### Step 7: Configure CloudWatch

Create alarms for CPU utilization (for example above 50%) so that the Auto Scaling Group scales out under load and scales in when load falls.

### Step 8: Test the Application

Open the DNS name of the load balancer in a browser and refresh repeatedly. The response should alternate between the two servers.

## 15. System Testing

Testing ensures the system works correctly. The first three test cases were executed and recorded; the last two describe the expected behaviour of the Auto Scaling Group.

### Test Case 1 – Target Registration and Health Check

Register both servers in the Target Group HA-Project-TG.

**Expected Result:**

Targets move from Initial to Healthy.

**Observed Result:**

Both targets (WebServer-1 in us-east-2a and WebServer-2 in us-east-2b) became Healthy (Figures 2–4).

### Test Case 2 – Load Balancer Creation

Create the Application Load Balancer and attach the listener.

**Expected Result:**

Load balancer becomes Active and gives a public DNS name.

**Observed Result:**

HA-Project-ALB changed from Provisioning to Active and showed its DNS name and listener (Figures 5–8).

## Test Case 3 – Load Balancing Across Availability Zones

Open the load balancer DNS name in a browser and refresh repeatedly.

**Expected Result:**

Responses alternate between the two servers.

**Observed Result:**

The page alternated between “Hello from Server 1 – AZ-A” and “Hello from Server 2 – AZ-B” (Figures 9–10).

## Test Case 4 – Instance Failure

Stop an EC2 instance.

**Expected Result:**

The target becomes Unhealthy, traffic goes to the healthy server and Auto Scaling launches a new instance.

## Test Case 5 – High Traffic

Generate load on the system.

**Expected Result:**

CPU alarm triggers and new instances are created; they are removed when load falls.

# 16. Results and Discussion

The screenshots below were captured from the working deployment in the AWS Management Console (US East – Ohio, us-east-2). The deployment has **WebServer-1** (us-east-2a) and **WebServer-2** (us-east-2b) registered in the target group **HA-Project-TG** behind the load balancer **HA-Project-ALB**. The account name and ID are blurred.

## 16.1 Target Group Registration and Health Checks

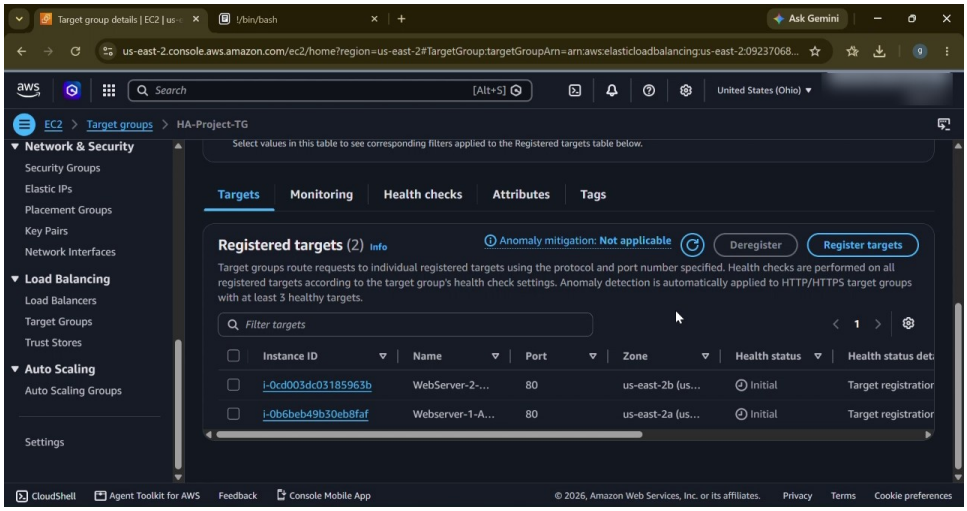



*Figure 2: Registered targets right after registration (status: Initial)*

**Explanation:** The target group HA-Project-TG lists two targets on port 80: instance i-0cd003dc03185963b (WebServer-2, us-east-2b) and i-0b6beb49b30eb8faf (WebServer-1, us-east-2a). The status Initial means the load balancer has just registered them and is still running the first health checks. No traffic is sent to a target until it passes.

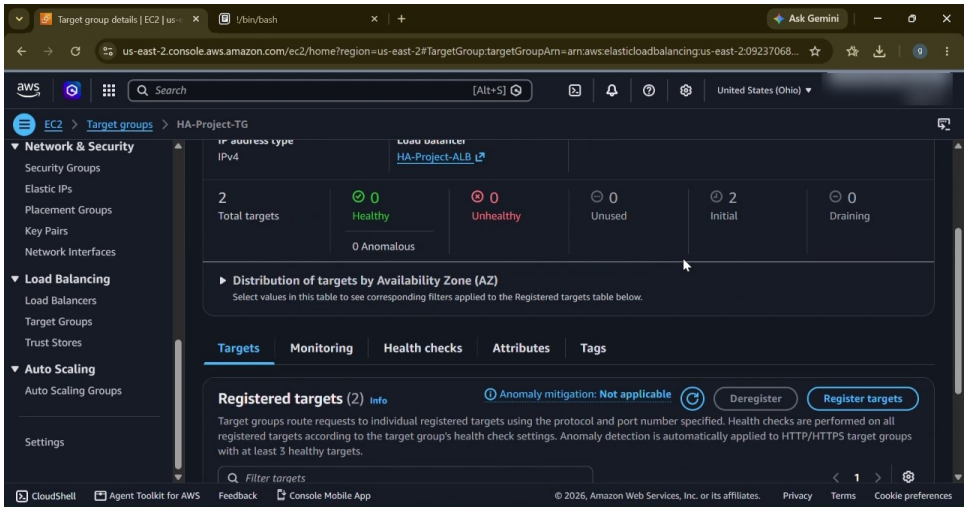



*Figure 3: Target group summary: 2 total targets, 0 healthy, 2 initial*

**Explanation:** The summary shows Total targets 2, Healthy 0, Unhealthy 0, Initial 2. The group is linked to the load balancer HA-Project-ALB. This is the starting state before the health checks complete.

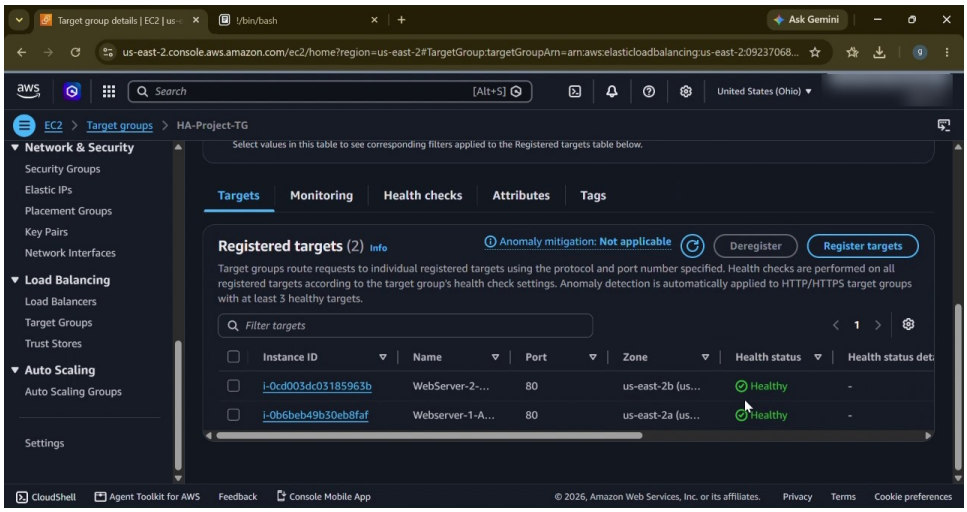



*Figure 4: Both targets reported Healthy*

**Explanation:** After a few health-check cycles both instances turn Healthy (green tick). This proves Nginx on each server answers the health check on port 80 and that the servers sit in two different Availability Zones, so there is no single point of failure. The load balancer now sends traffic to both.

## 16.2 Application Load Balancer

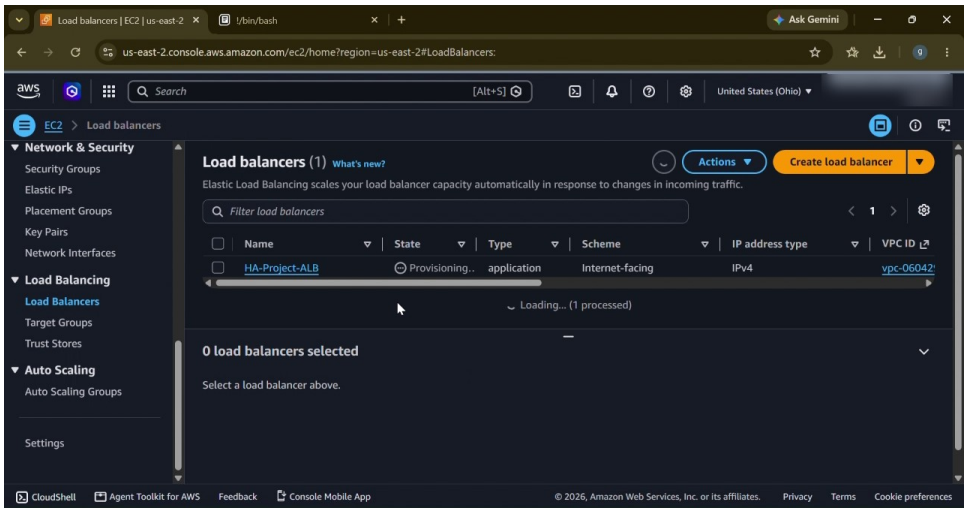



*Figure 5: HA-Project-ALB in Provisioning state*

**Explanation:** HA-Project-ALB is an application load balancer, internet-facing, using IPv4. AWS is still creating its nodes in the selected subnets.

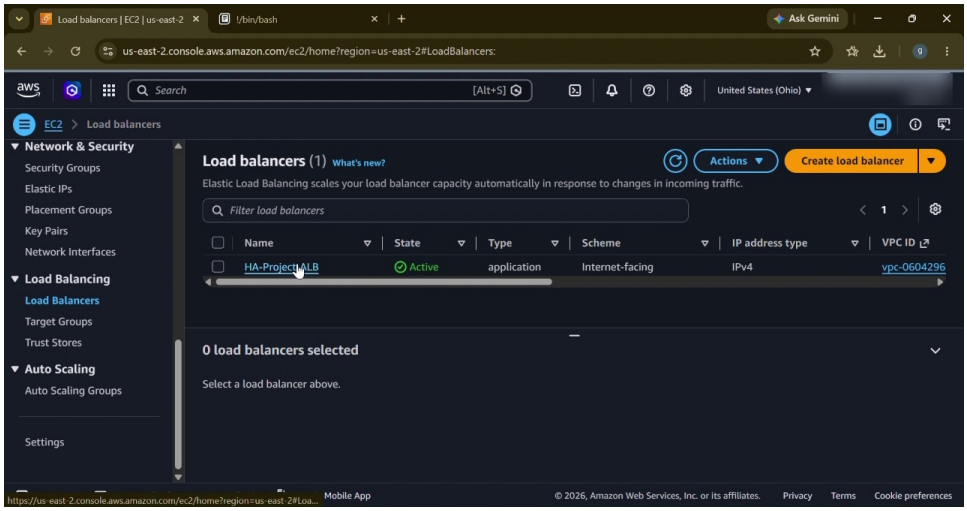



*Figure 6: HA-Project-ALB in Active state*

**Explanation:** After provisioning completes, the state changes to Active. Only an active load balancer can accept and forward requests.

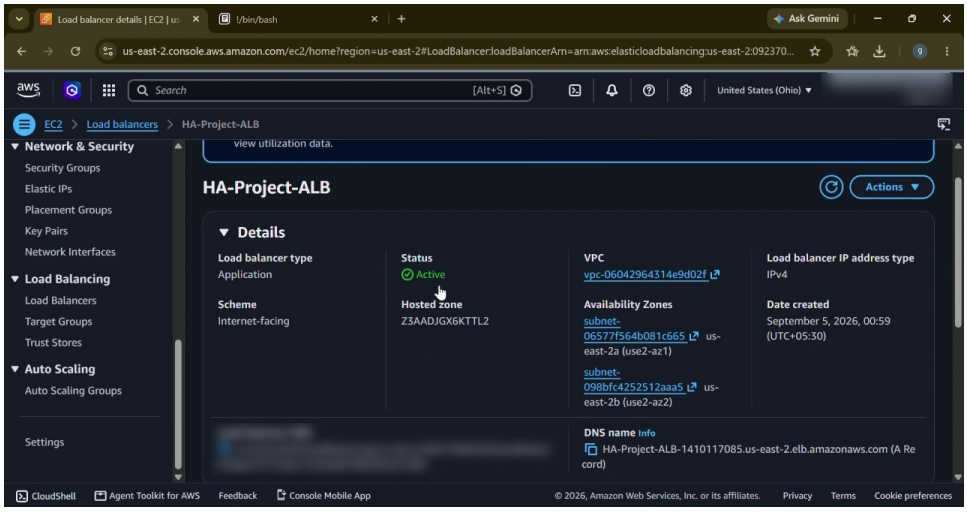



*Figure 7: Details of HA-Project-ALB*

**Explanation:** Type Application, scheme Internet-facing, status Active, IPv4, VPC vpc-06042964314e9d02f, created on 5 September 2026. It spans two Availability Zones, us-east-2a and us-east-2b, each with its own subnet, which lets the system survive the loss of a whole zone. Its DNS name (HA-Project-ALB-1410117085.us-east-2.elb.amazonaws.com) is the public address of the application.

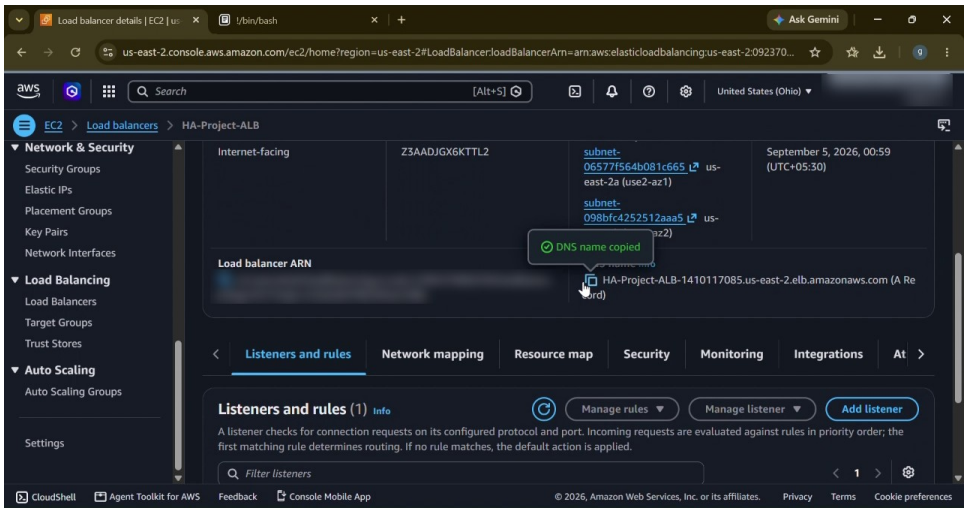



*Figure 8: DNS name copied; Listeners and rules (1)*

**Explanation:** The DNS name is copied so it can be pasted into a browser. The Listeners and rules tab shows one listener that accepts HTTP and forwards requests to the target group HA-Project-TG.

## 16.3 Application Output through the Load Balancer

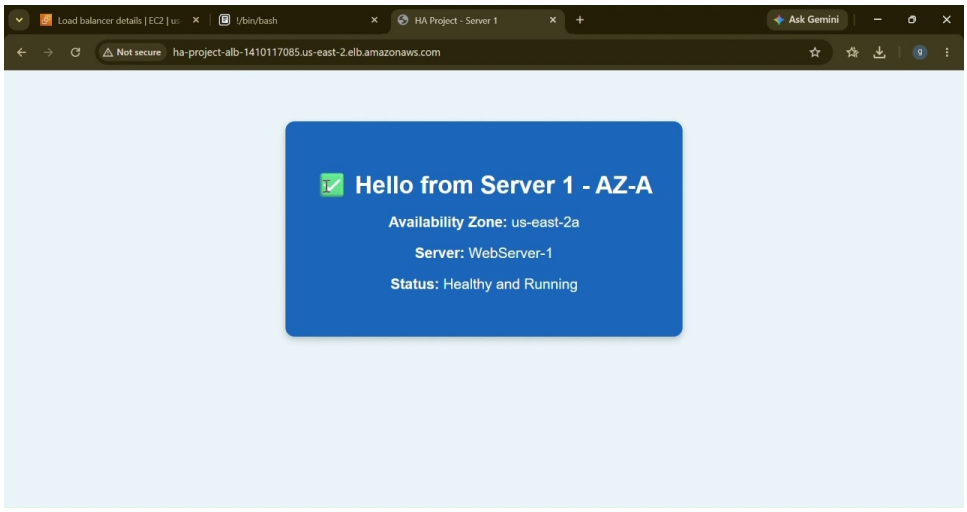



*Figure 9: Browser output: Hello from Server 1 – AZ-A*

**Explanation:** Opening the load balancer DNS name returns the page served by WebServer-1: "Hello from Server 1 – AZ-A", Availability Zone us-east-2a, Status: Healthy and Running. The browser shows "Not secure" because only HTTP is used in this demo. The URL is the load balancer, not an individual server.

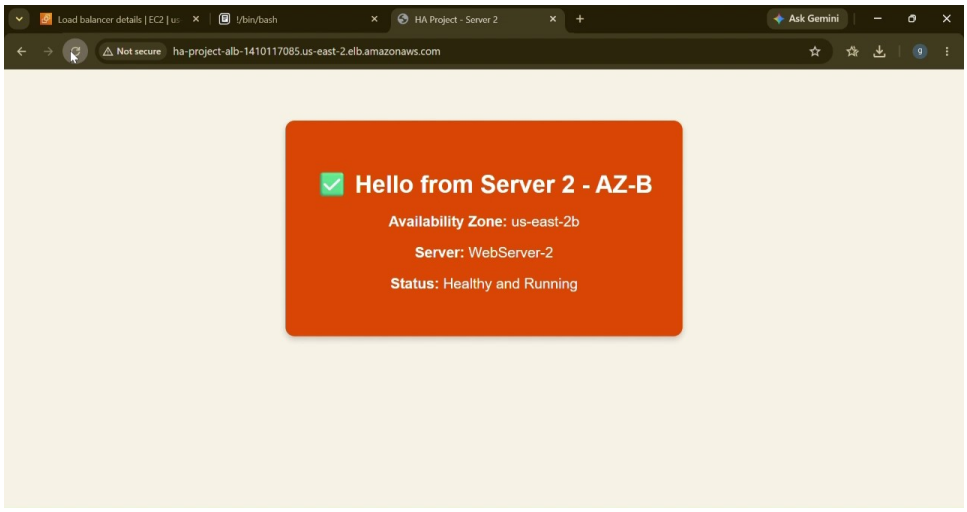



*Figure 10: Browser output: Hello from Server 2 – AZ-B*

**Explanation:** On refreshing the same URL, the response comes from the other server: “Hello from Server 2 – AZ-B”, Availability Zone us-east-2b, WebServer-2. The page colour also changes (blue for Server 1, orange for Server 2), which makes the switching easy to see.

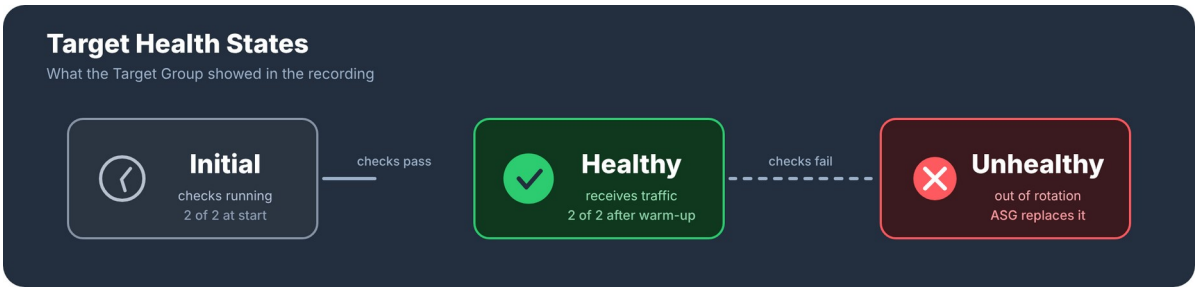



*Figure 11: Target health states*

## 16.4 Discussion

The system successfully demonstrates:

- **Load balancing** – the same load balancer URL answered alternately from Server 1 and Server 2
- **High availability** – the servers run in two Availability Zones
- **Health checking** – only Healthy targets receive traffic; a failed server is removed from rotation and replaced by the Auto Scaling Group

*The infrastructure ensures high availability and improved performance.* The Auto Scaling Group, CloudWatch alarms and S3/IAM parts are part of the design described in this report; the recorded demonstration covers the Target Group, the Application Load Balancer and the web-server output.

## 16.5 Summary of Observations

- **Registration:** both targets entered the Target Group in the *Initial* state and moved to *Healthy* after the first health checks.
- **Distribution:** the two servers sit in different Availability Zones (us-east-2a and us-east-2b), so the loss of one zone does not stop the service.
- **Entry point:** the load balancer reached the *Active* state and exposed a single DNS name that users open in the browser.
- **Traffic sharing:** repeated refreshes of the same URL returned Server 1 and Server 2 alternately.
- **Security:** the account details are hidden in the screenshots, and the demo uses plain HTTP only.

These observations confirm that the application tier is *redundant*, *load balanced* and *health checked*, which are the foundations on which the self-healing and auto-scaling behaviour is built.

# 17. Advantages

- **High system availability** – servers in two Availability Zones remove the single point of failure.
- **Self-healing** – failed instances are detected and replaced with no manual intervention.
- **Automatic scaling** – capacity grows under load and shrinks when demand falls.

- **Reduced downtime** – traffic is moved away from unhealthy servers immediately.
- **Cost optimization** – you pay only for the instances that are needed at a given time.
- **Security** – Security Group chaining allows web traffic only through the load balancer, and the IAM Role avoids stored access keys.
- **Automated, repeatable management** – the Launch Template builds every server in exactly the same way.

*These advantages make the design suitable for real-world web applications that must stay online.*

# 18. Limitations

- **Requires AWS knowledge** – the services, networking and security settings must be understood to configure them correctly.
- **Dependent on cloud provider** – the solution is tied to AWS services and their availability.
- **Initial configuration complexity** – many components (VPC, Security Groups, load balancer, target group, scaling group) must be set up in the right order.
- **Scaling is not instant** – new instances need time to boot and pass health checks, and a cooldown period applies.
- **Cost if left running** – the load balancer and extra instances are charged hourly, so resources should be deleted after the demonstration.

- **HTTP only in the demo** – HTTPS and a custom domain are not configured.

*Most of these limitations can be reduced with careful threshold tuning, infrastructure-as-code and HTTPS certificates, as discussed under Future Scope.*

# 19. Applications

This system can be used for:

- **E-commerce websites** – stay available and fast during sales and festival traffic.
- **Online learning platforms** – handle many students joining classes or exams at the same time.
- **Enterprise applications** – provide reliable internal and customer-facing web services.
- **Financial systems** – keep services running with minimal downtime.
- **SaaS applications** – scale with the number of customers automatically.

Any stateless web application that needs high availability and variable capacity can use the same architecture.

# 20. Future Scope

Future improvements include:

- **HTTPS and custom domain** – use AWS Certificate Manager and Route 53.

- **Managed database** – add Amazon RDS / Aurora Multi-AZ for stateful applications.
- **Stronger security** – private subnets with NAT, CloudFront and AWS WAF.
- **Better scaling policies** – target-tracking on request count and SNS email alerts.
- **Containers and serverless** – Kubernetes container orchestration and AWS Lambda.
- **Multi-region deployment** – survive the failure of a whole region.
- **Intelligent scaling** – AI-based prediction of traffic.
- **Infrastructure as code** – Terraform / CloudFormation with a CI/CD pipeline.

# 21. Conclusion

This project demonstrates how cloud computing technologies can be used to build a scalable and reliable infrastructure.

The integration of **EC2, Application Load Balancer, Auto Scaling, and CloudWatch** enables the system to automatically handle failures and dynamically scale resources based on traffic demand. The captured output confirms that two web servers in different Availability Zones registered in a target group, passed their health checks and served the application alternately through a single load balancer address.

Through this work, practical experience was gained in designing a multi-AZ architecture, configuring an Application Load Balancer and Target Group, understanding health checks, and planning automatic scaling with CloudWatch. These are core skills for cloud and DevOps roles.

*The implementation highlights the importance of automation in cloud infrastructure, ensuring improved reliability, performance, and cost efficiency.*

# 22. References


1. AWS Documentation
   https://aws.amazon.com/documentation
2. Amazon EC2 User Guide
   https://docs.aws.amazon.com/ec2
3. Elastic Load Balancing User Guide
   https://docs.aws.amazon.com/elasticloadbalancing
4. Amazon EC2 Auto Scaling User Guide
   https://docs.aws.amazon.com/autoscaling
5. Amazon CloudWatch Documentation
   https://docs.aws.amazon.com/cloudwatch
6. Nginx Documentation
   https://nginx.org/en/docs